

Riesgos de HTML/JS en recursos educativos — informe completo

Ernesto Serrano Collado · info@ernesto.es

2026-06-14

Índice

Resumen	2
1. Introducción	2
2. Background	3
3. Metodología	4
4. Resultados	5
5. Discusión	8
6. Mitigaciones	9
7. Limitaciones	10
8. Ética y divulgación responsable	11
9. Declaración de conflicto de interés	11
10. Disponibilidad de artefactos	11
11. Conclusiones	11
12. Declaración sobre el uso de IA generativa	12
13. Referencias y estándares	12
Anexo A — Matriz de seguridad	13
1. Tabla resumida (para el cuerpo del artículo)	14
2. Matriz técnica completa (anexo)	15
3. Mitigaciones emparejadas (riesgo → mitigación → limitación)	17
4. Tabla de concienciación (perfil no técnico)	18
Anexo B — Anexos técnicos	18
A. Metodología	18
B. La sonda (<code>probe.js</code>) — redacted	18
C. Resultados por plataforma	19
D. Resultados por navegador	19
E. Comportamiento del navegador (referencia)	20
F. Compatibilidad	20
G. Mitigaciones emparejadas (riesgo → mitigación → limitación)	20
H. Limitaciones metodológicas	20
I. Trabajo futuro	21
J. Notas de seguridad de esta investigación (checklist cumplido)	21
Anexo — Confirmación en vivo en una instalación de Moodle en línea (2026-06-13)	21

Moodle, WordPress, Omeka S, SCORM, H5P y eXeLearning

Ernesto Serrano Collado · Independent Researcher, España · ORCID: 0009-0006-3817-1317

Documento a título personal. Declaración de conflicto de interés: el autor colabora en el proyecto eXeLearning y es autor/mantenedor de varias piezas evaluadas (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`); el modo seguro descrito en la sección 6.2 es aportación propia. Véase la tabla de procedencia en la sección 9.

Resumen

Los recursos educativos interactivos —SCORM, H5P, paquetes de eXeLearning, páginas HTML— a menudo necesitan JavaScript para funcionar. El problema no es JavaScript: es ejecutar JavaScript no confiable dentro de una sesión autenticada del LMS sin una frontera de aislamiento explícita. Este trabajo es una **sistematización (SoK) y evaluación empírica de seguridad** de ocho formas habituales de publicar contenido en Moodle, WordPress y Omeka S (en sus versiones estables), más las tres variantes mitigadas de las integraciones mantenidas, realizada con el código fuente delante y con una sonda de capacidades inocua ejecutada en laboratorio local. El hallazgo central, organizado en una matriz comparativa con un mismo modelo mental (¿ejecuta JS de autor?, ¿en el mismo origen que la plataforma?, ¿hay aislamiento real?), es que cuando el contenido corre en el mismo origen que la plataforma puede leer el DOM autenticado, los formularios con *token* y aprovechar la sesión; cuando corre aislado (origen opaco, `sandbox` sin `allow-same-origin`), no.

Distinguimos tres estados con precisión: (a) las *versiones estables analizadas* de las integraciones de eXeLearning y el SCORM nativo de Moodle ejecutan el contenido en el mismo origen; (b) las *integraciones de eXeLearning mantenidas* (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`) incorporan un modo seguro de origen opaco activado por defecto que cierra esa exposición; (c) `mod_page`, el SCORM nativo y `mod_exeweb/mod_exescorm` siguen siendo *same-origin* por diseño. H5P es un caso aparte: no ejecuta HTML de autor, sino librerías curadas. Sobre `mod_page` precisamos un punto que la literatura suele confundir: su protección real no es el saneamiento *server-side* (usa `noclean=true`), sino la restricción de capacidad/rol (`mod/page:addinstance`).

La IA generativa actúa como factor de frecuencia —facilita pegar HTML/JS sin revisar—; no la medimos empíricamente, la usamos como motivación y contexto de amenaza. La mitigación efectiva vive en el servidor y en las cabeceras, no en confiar en que el contenido “no encuentre” el *token*. Mostramos, y verificamos en navegador (Chromium y Firefox 146/Gecko), un patrón de *hardening* —origen opaco + puente `postMessage` validado + CSP— que mantiene interactividad y *tracking* sin tratar el contenido como parte de la plataforma. A lo largo del texto separamos el estado externo evaluado (versiones estables, propias y de terceros), la mitigación de aportación propia (el modo seguro, sección 6.2; tabla de procedencia en la sección 9) y las propuestas de trabajo futuro (sección 6.3).

Tesis: el riesgo principal de los recursos educativos interactivos no es JavaScript, sino ejecutar JavaScript **de autor** dentro del mismo origen autenticado del LMS. Un modelo basado en origen opaco, `sandbox` estricto, CSP y puente `postMessage` validado permite mantener la interactividad sin confiar en el contenido como si fuera parte de la plataforma.

Palabras clave: seguridad web; LMS; Moodle; eXeLearning; SCORM; H5P; WordPress; Omeka S; aislamiento de `iframe`; política de mismo origen; `sandbox`; Content Security Policy; `postMessage`; XSS; contenido generado por IA.

1. Introducción

Hoy es trivial pedirle a un asistente de IA “hazme una actividad interactiva en HTML” y pegar el resultado en un recurso del LMS. Ese HTML puede traer `<script>`, `<iframe>`, `onerror=...`, `fetch()` o librerías de terceros que nadie ha revisado —un riesgo de confianza transitiva medido a gran escala en la web [1] y agravado por librerías desactualizadas con vulnerabilidades conocidas [2]—. A la vez, plantillas de foros, *snippets* de StackOverflow, embeds de redes sociales y *trackers* de analítica se copian y pegan tal cual. Un paquete subido por una persona autora —interna o externa— que se renderiza dentro de la sesión de una persona docente o con rol de administración hereda, si no hay aislamiento, parte de los privilegios de esa sesión. La literatura ya señala el contenido y los recursos como vector de riesgo en sistemas de *e-learning* [3] y en aplicaciones de aprendizaje en línea en general [4]; análisis empíricos recientes de vulnerabilidades en LMS (Moodle, Chamilo, ILIAS) lo corroboran [5]. En el ámbito de los Recursos Educativos Abiertos, CEDEC/INTEF advierte que pegar código generado por IA sin poder explicarlo hace “perder la A de Abierto” y puede ocultar funcionalidad insegura [6].

La distinción clave es entre interactividad legítima y código no confiable: una simulación física o un cuestionario necesitan JS; el riesgo aparece cuando ese JS proviene de una fuente que el LMS trata como de confianza sin haberla verificado.

Encuadramos el trabajo como una **sistematización (SoK) y evaluación empírica de seguridad** —un *estudio*

de diseño—, no como un reporte de vulnerabilidades: el valor está en la comparación sistemática entre mecanismos y en el patrón de mitigación, no en hallazgos aislados (la naturaleza del problema se acota más abajo).

Pregunta de investigación. ¿Hasta qué punto las formas habituales de publicar recursos educativos interactivos en Moodle, WordPress y Omeka S aíslan el JavaScript de autor respecto a la sesión autenticada de la plataforma?

Subpreguntas:

- **RQ1.** ¿Qué integraciones ejecutan JavaScript de autor?
- **RQ2.** ¿Cuáles lo ejecutan en el mismo origen que la plataforma?
- **RQ3.** ¿Qué mitigaciones permiten mantener interactividad y *tracking* sin exponer el DOM autenticado?
- **RQ4.** ¿Cómo cambia la IA generativa el *modelo de amenaza operativo* de estos recursos? (factor de plausibilidad y frecuencia, **no medido** empíricamente.)

Contribuciones. (1) Una **sistematización** del riesgo de contenido educativo interactivo en tres ejes —¿ejecuta JS de autor?, ¿en el mismo origen que la plataforma?, ¿hay aislamiento real?— con un criterio de clasificación explícito (sección 3.4); (2) una evaluación empírica de ocho mecanismos de publicación (en sus versiones estables, más tres variantes mitigadas mantenidas) en Moodle, WordPress y Omeka S, anclada a **archivo:línea** y verificada en laboratorio, con una tabla *afirmación* → *evidencia* (sección 4.1); (3) un conjunto de PoC inocuas y reproducibles (solo booleanos, sin *payloads* reutilizables); (4) un patrón de mitigación —origen opaco + **sandbox** estricto + CSP + puente **postMessage** validado— compatible con la interactividad y el *tracking* SCORM, separado en requisitos de diseño (sección 6.2.1) e implementación de referencia (sección 6.2.2); y (5) una discusión —no una medición— sobre el efecto de la IA generativa y los Recursos Educativos Abiertos (REA).

Naturaleza del hallazgo (no es un 0-day). Conviene situar el tipo de problema. Distinguimos cuatro categorías: (a) *vulnerabilidad* —un fallo concreto y corregible (p. ej. un XSS evadible)—; (b) *riesgo por diseño* —comportamiento esperado y documentado, pero peligroso si el contenido no es de confianza (el *same-origin* de SCORM, `noclean=true` en `mod_page`)—; (c) *mala configuración* —capacidades o roles demasiado amplios—; y (d) *cadena de suministro* —librerías H5P, JS de terceros o contenido generado por IA—. Este artículo **no** reivindica *0-days* de terceros: evalúa **fronteras de confianza** en la publicación de recursos educativos y, en particular, la **ausencia de una frontera de origen** entre el contenido de autor y la sesión autenticada del LMS.

2. Background

Política de mismo origen (SOP). El navegador aísla documentos por *origen* (esquema + host + puerto). Un script solo puede leer el DOM, las cookies o el almacenamiento de documentos de su mismo origen; el acceso entre orígenes distintos lanza `SecurityError`. La SOP es la frontera que decide si el contenido de un recurso puede “ver” la sesión del LMS.

iframe sandbox. El atributo `sandbox` restringe lo que un `iframe` puede hacer; los *tokens* (`allow-scripts`, `allow-same-origin`, `allow-popups`, `allow-forms`, `allow-top-navigation`...) reactivan capacidades concretas. Punto crítico: combinar `allow-scripts` con `allow-same-origin` sobre contenido del propio origen **anula el aislamiento** —el propio navegador advierte de que el contenido puede “escapar”— [7], [8]. Sin `allow-same-origin`, el documento obtiene un **origen opaco** (`null`) y queda aislado del padre. Además, **los flags de sandbox se propagan a los iframes anidados**: un reproductor de YouTube embebido dentro de un `iframe` opaco hereda la restricción y pierde su propio origen.

Content Security Policy (CSP). Cabecera que limita de qué orígenes puede el documento cargar scripts, imágenes, marcos o hacer conexiones (`script-src`, `img-src`, `frame-src`, `connect-src`, `frame-ancestors`, `object-src`, `base-uri`, `sandbox`...) [9]. Las *whitelists* de CSP son frágiles; se recomiendan estrategias basadas en *nonce/strict-dynamic* [10], y los análisis longitudinales muestran que **desplegar una CSP efectiva en producción es difícil** [11].

SCORM, H5P, eXeLearning. SCORM define que el contenido (el *SCO*) se comunique con el LMS por un objeto JavaScript (`window.API` en 1.2, `API_1484_11` en 2004) descubierto recorriendo `window.parent` [12], [13], [14]. H5P renderiza *tipos de contenido* (librerías) curados por la administración, no HTML de autor [15]. eXeLearning exporta paquetes con HTML/JS del autor (`.elpx`) que las integraciones embeben en un `iframe`.

3. Metodología

3.1 Plataformas y versiones

Analizamos: `mod_exelearning`, `mod_exeweb`, `mod_exescorm`, SCORM nativo (`mod_scorm`), H5P (`mod_h5pactivity` / `core_h5p`), el recurso *Página* (`mod_page`), `wp-exelearning` y `omeka-s-exelearning`. Las versiones estables analizadas (las que fijan el “estado actual” de cada plataforma) son: **Moodle 5.0.7** (núcleo 2104c372962) con `mod_exelearning` 2c5473d, `mod_exeweb` 60d24fb, `mod_exescorm` e985f4d y el editor eXeLearning 8101f54e; **WordPress** (vía `wp-env`) con `wp-exelearning`; y **Omeka S** (Docker, imagen `erseco/alpine-omeka-s:develop`) con `omeka-s-exelearning`. La matriz completa por archivo:línea está en `matriz-seguridad.md`; los anexos por plataforma y navegador en `anexos-tecnicos.md`.

Nota de estado. El **modo seguro de origen opaco** descrito en la sección 6.2 es el **diseño por defecto** de las integraciones mantenidas (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`), distinto del comportamiento *same-origin* de las versiones estables que aquí se analiza como “estado actual”. El artículo separa explícitamente ambos.

3.1.1 Criterios de selección del corpus

Seleccionamos los mecanismos con tres criterios: (i) **prevalencia** —formas habituales de publicar contenido de autor en Moodle, WordPress y Omeka S—; (ii) **ejecución o *embedding* de HTML/JS de autor** (excluimos mecanismos que no corren código de autor, como recursos de solo texto o enlaces); y (iii) **cobertura del espectro de confianza**: desde *sin aislamiento* (ventana top: `mod_page`; `iframe` sin `sandbox`: SCORM nativo, `mod_exeweb`, `mod_exescorm`), pasando por *filtrado semántico* (H5P), hasta *aislamiento por origen opaco configurable* (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`). El conjunto es **representativo** de las tres relaciones contenido origen que determinan el riesgo (top *same-origin*, `iframe same-origin`, `iframe opaco`), no una muestra exhaustiva de todos los plugins existentes; quedan fuera, por tanto, los recursos que no ejecutan JS de autor y las plataformas no incluidas en el estudio.

3.2 Entorno

Instancias **Docker locales y desechables**; curso/página de laboratorio marcados POC-SAFE. Las pruebas vivas en navegador se realizaron con **dos motores**: uno basado en Chromium y **Firefox (Gecko 146)** vía Playwright. Verificamos en vivo `mod_exelearning`, `mod_scorm`, `mod_h5pactivity`, `mod_page`, `wp-exelearning` y `omeka-s-exelearning`; el resto (`mod_exeweb`, `mod_exescorm`) se verificó sobre código fuente.

Verificación cross-browser. Para confirmar que el aislamiento de origen opaco es un **comportamiento definido por el estándar web** y no de un motor concreto, replicamos la comprobación en **Firefox 146 (Gecko)** con un script de Playwright reproducible (`evidencias/firefox-isolation-test.cjs`). (i) En una prueba auto-contenida, un `iframe` con `sandbox con allow-same-origin` lee `window.parent` (origen heredado), mientras que *sin* `allow-same-origin` el acceso lanza `SecurityError` e `isOpaqueOrigin` es `true` —medido **desde dentro** del propio `iframe`—. (ii) Los `embeds` reales en modo seguro de `mod_exelearning` (servido por `tokenpluginfile`), `wp-exelearning` y `omeka-s-exelearning` resultan **opacos** también en Firefox: `contentDocument === null` y `contentWindow` lanza `SecurityError` en las **tres** integraciones. El resultado es **idéntico al de Chromium** (`evidencias/resultados-firefox.json`, `evidencias/resultados-firefox-moodle.json`).

3.3 La sonda (*probe.js*): definición de cada medida

Las pruebas de concepto comparten una sonda que ejecuta una serie de comprobaciones y **solo** devuelve booleanos y nombres de error *redacted*. Nunca lee valores reales, nunca hace `red`, nunca hace `POST`, nunca invoca mutadores SCORM. Cada medida:

Booleano	Qué detecta (no ejerce)
<code>canRunJavascript</code>	el navegador ejecuta el script del autor
<code>isOpaqueOrigin</code>	el documento corre en origen opaco (<code>null</code>)
<code>sandboxAttr</code> / <code>sandboxAllowsSameOrigin</code>	atributo <code>sandbox</code> efectivo y si concede <code>allow-same-origin</code>
<code>canAccessParent</code> / <code>canReadParentDocument</code>	si <code>window.parent/parent.document</code> son accesibles (mismo origen)
<code>canReadParentCookie</code>	si <code>parent.document.cookie</code> es legible (valor siempre <code>REDACTED</code>)
<code>canFindSesskey</code>	si el <code>sesskey/nonce</code> está presente en el DOM <code>same-origin</code> (valor <code>REDACTED</code>)
<code>canFindCourseEditForms</code> / <code>canFindCourseEditLinks</code>	presencia de formularios/enlaces de edición

Booleano	Qué detecta (no ejerce)
canAccessTop / canAttemptTopNavigation	alcanzabilidad de la ventana top (no navega; <code>not_attempted</code>)
canOpenPopups	abre y cierra de inmediato un popup 1×1 (inocuo)
canUsePostMessage / canPostMessageToParent	disponibilidad del canal (no envía nada)
canCallScormApi / scormApiFlavor	si <code>window.API/API_1484_11</code> es alcanzable (no lo invoca)
canUseLocalStorage / canUseSessionStorage	acceso a almacenamiento same-origin
sandboxEscape	siempre false: detectado por diseño, nunca intentado

Cuatro paquetes encapsulan la sonda: `evil.elpx` (eXeLearning), `evil.h5p` (control negativo: H5P la filtra), `evil-scorm.zip` (SCORM 1.2 que **detecta** `window.API`) y `evil-page.html` (sonda *inline* para *Página*). Salida típica *redacted*:

```
{ "canRunJavascript": true, "canAccessParent": false, "canReadParentDocument": false,
  "canReadParentCookie": false, "parentCookieValue": "REDACTED", "canFindSesskey": false,
  "sesskeyValue": "REDACTED", "canCallScormApi": true, "isOpaqueOrigin": true,
  "sandboxEscape": false, "error": "SecurityError" }
```

3.4 Criterio de clasificación de riesgo

Clasificamos cada mecanismo con un criterio explícito sobre **siete dimensiones** observables: (i) ¿ejecuta JS de autor?; (ii) ¿en el mismo origen que la plataforma?; (iii) rol mínimo para *publicar* el contenido; (iv) rol del *visitante* que lo ejecuta; (v) ¿hay un *token* de sesión legible en el DOM *same-origin*?; (vi) ¿se emite CSP restrictiva?; (vii) ¿existe una capacidad mutadora *server-side* alcanzable? A partir de ellas separamos el *impacto máximo demostrado* (verificado en laboratorio) del *impacto inferido* (deducido del modelo del navegador). El resumen divulgativo en tres niveles:

- **Bajo:** no ejecuta JS de autor, o lo ejecuta en un origen aislado (*opaco/srcdoc*) sin acceso al padre.
- **Medio:** ejecuta JS aislado pero con canales residuales (*popups*, *postMessage*), o filtrado por semántica evadible (XSS documentados).
- **Alto:** ejecuta JS de autor en el mismo origen que el LMS (acceso al DOM/sesión autenticada), o en la ventana top sin sandbox.

La **matriz de riesgo formal** por plataforma —estas siete dimensiones más el impacto demostrado/inferido— está en `matriz-seguridad.md` (sección 1).

3.5 Límites éticos del método

No robamos cookies ni *tokens*, no exfiltramos nada y no atacamos sistemas externos. La sonda distribuida (`probe.js`) **no hace red ni POST**: solo **detecta** capacidades y muestra una tabla de booleanos *redacted*. Para **confirmar el impacto** (sección 4.2 y anexos) sí ejecutamos, de forma **autorizada y reversible**, peticiones reales —incluidos POST— **únicamente sobre cuentas propias o de laboratorio**, nunca destructivas, nunca en producción y nunca contra terceros; los cambios (p. ej. el nombre/foto del propio perfil) se revirtieron. No publicamos *payloads* reutilizables. Detalle en la sección 8 (Ética y divulgación responsable).

4. Resultados

4.1 Tabla principal

Plataforma / recurso	¿Ejecuta JS del autor?	¿Mismo origen que el LMS?	Aislamiento real	Nivel de riesgo
<code>mod_page</code> (Página)	Sí (<code>noclean=true</code> ; verificado)	Sí (ventana top, sin <i>iframe</i>)	Ninguno <i>server-side</i> ; solo <i>gated</i> por <code>mod/page:addinstance</code>	Alto si lo edita el profesorado (corre en la sesión de cada visitante)
<code>mod_scorm</code> (core) <code>mod_h5pactivity</code> / <code>core_h5p</code>	Sí Parámetros: No (filtra). Librerías: Sí (<code>preloadJs</code> , código de confianza)	Sí Sí	Ninguno (sin sandbox) Parámetros filtrados por semántica; el JS de las librerías corre <i>same-origin</i> sin sandbox	Alto Bajo en contenido; alto si se pueden instalar librerías (<code>h5p:updatelibraries</code> , gestión/administración)
<code>mod_exelearning</code>	Sí	estable/legacy: Sí mantenido (<code>secure</code> , def.): No (opaco)	Fuerte en <i>secure</i> (origen opaco + bridge), parcial en <i>legacy</i>	Bajo en <i>secure</i> / medio-alto en <i>legacy</i>
<code>mod_exeweb</code>	Sí	Sí	Ninguno	Alto

Plataforma / recurso	¿Ejecuta JS del autor?	¿Mismo origen que el LMS?	Aislamiento real	Nivel de riesgo
mod_exescorm	Sí	Sí	Ninguno	Alto
wp-exelearning	Sí	estable/legacy: Sí · mantenido (secure, def.): No (opaco)	Fuerte en secure (origen opaco), parcial en legacy	Bajo en secure / medio-alto en legacy
omeka-s-exelearning	Sí	estable/legacy: Sí · mantenido (secure, def.): No (opaco)	Fuerte en secure (origen opaco), parcial en legacy	Bajo en secure / medio-alto en legacy

Lectura rápida (responde a RQ1–RQ2): ejecutar JavaScript de autor no es el problema; el problema es ejecutarlo con el mismo origen que el LMS y sin una frontera explícita.

4.1.1 Afirmación → evidencia

Para que el lector distinga sin ambigüedad qué está verificado en vivo, qué en código y qué queda pendiente:

Afirmación	Tipo de evidencia	Fuente
mod_page ejecuta <script> de autor	Vivo (Moodle 5.0.7 local) + código	§4.2; mod/page/view.php:90–93
SCORM nativo corre <i>same-origin</i> sin sandbox	Vivo + código	§4.3; player.php:279–285
H5P filtra los <i>parámetros</i> (no ejecutan)	Vivo (control negativo)	§4.4; poc/evil.h5p
El <i>preloadedJs</i> de una librería H5P corre <i>same-origin</i>	Código + PoC validada + procedimiento manual	§4.4; resultados-h5p-library.json
El modo seguro aísla (origen opaco)	Vivo en Chromium y Firefox 146/Gecko	§4.5–4.6; resultados-firefox*.json
mod_exeweb / mod_exescorm <i>same-origin</i> sin sandbox	Solo código (inferencia)	matriz §2.2
Autoedición persistente del propio perfil (legacy)	Vivo, autorizado y reversible (cuenta propia)	anexo (Confirmación en vivo)
Safari / WebKit	No verificado (trabajo futuro)	—

4.2 mod_page (Página) — la protección es la capacidad, no el saneamiento

Un usuario con la capacidad de crear una Página escribe HTML. Al mostrarse, `mod_page` llama a `format_text(..., noclean=true)` (`mod/page/view.php:90–93`, `lib.php:352`). Creamos una Página con `<script>` y `<img onerror>` y la abrimos: **ambos se ejecutaron**. Con `noclean=true` (y `forceclean=0` por defecto), `format_text` **no** pasa por `purify_html()`; el `<script>` del autor se almacena y se ejecuta para **quien lo visualice (incluido el alumnado)**, en la **ventana principal** y en el **mismo origen** que Moodle.

La protección real, por tanto, **no es el filtrado server-side** —no hay— **sino la capacidad/rol**: solo roles autorizados pueden crear/editar una Página (`mod/page:addinstance`). El flag `$CFG->enabletrusttext` está desactivado por defecto, pero **no** condiciona la ejecución en `mod_page`, porque este usa `noclean`. Corregimos así una formulación habitual: la defensa no es “Moodle filtra `<script>`”, sino “solo una persona docente o gestora puede publicar la Página”.

Impacto en la sesión de una persona estudiante (verificado en código). El script —*same-origin*, ventana top— puede: (a) no leer la cookie de sesión (MoodleSession es `HttpOnly` por defecto), pero sí *aprovechar la sesión* leyendo `M.cfg.sesskey` y, como la página principal de Moodle no emite CSP restrictiva, exfiltrar el `sesskey` y datos del DOM y forjar peticiones autenticadas; (b) modificar el DOM de la vista y cambiar persistentemente su propio perfil (nombre/foto reenviando el formulario de `user/edit.php`, confirmado en un Moodle real); (c) enviar mensajes en su nombre —`core_message_send_instant_messages` es `'ajax' => true` con `moodle/site:sendmessage`, capacidad del rol `user`—, lo que habilita una *propagación dependiente de interacción* (el receptor debe abrir un enlace; el cuerpo del mensaje se sanea al mostrarse, así que no se auto-ejecuta).

Límite de autopropagación. Una persona estudiante **no** puede plantar HTML ejecutable: los foros usan `trusttext` (con `enabletrusttext` apagado se sanea) y carece de `addinstance`. La propagación **escala** si quien abre la Página es una persona docente o gestora: con sus privilegios el script puede crear más Páginas y llamar a servicios privilegiados (p. ej. `core_user_update_users`, `'ajax' => true`).

Alcance: confinado al origen, pero vector latente. Por la SOP, el script no puede leer cookies/DOM de otras webs de la persona usuaria (banca, correo), ni contraseñas guardadas, ni otras pestañas; el alcance se limita a la sesión del propio LMS. Aun así, disponer de JS arbitrario en el origen autenticado es un punto de apoyo persistente: podría aprovechar una vulnerabilidad futura alcanzable desde ese origen (un servicio sin chequeo de

capacidad, un IDOR/CSRF), y su impacto **queda acotado por las capacidades del rol que lo abre** (si lo abre un perfil de administración, ejecuta acciones de administración). Además, no necesita actuar de inmediato: el script puede permanecer latente —inofensivo para el alumnado— y condicionar su carga a quién abre la página (`M.cfg.userId/roles`, elementos del DOM visibles solo a perfiles de gestión, o sondeo de capacidades), de modo que solo se activa ante un rol privilegiado. Es, en el peor caso, un ataque dirigido y paciente, siempre **sujeto a las capacidades del visitante**.

El mismo canal de mensajería (`core_message_send_instant_messages`) permitiría además *inducir* a un perfil de mayor privilegio a abrir el recurso, sujeto a la política de mensajería: por defecto (`messagingallusers=0`) solo a contactos/compañeros de curso, y a cualquiera solo si `messagingallusers` está activado. El impacto, una vez abierto, queda **acotado por las capacidades de ese perfil**: con creación de cursos o gestión podría crear un curso (lo reproducimos *scrapeando* `course/edit.php`) y matricular alumnado (`enrol_manual_enrol_users`, en los cursos que gestiona); un perfil de administración podría modificar cuentas o configuración del sitio. Es decir, el escalado depende del rol y de la configuración; no es automático.

El origen opaco elimina ese punto de apoyo; `mod_page` no lo tiene.

4.3 SCORM nativo (`mod_scorm`)

El SCO recorre `window.parent/window.opener` buscando `window.API` (`mod/scorm/loadSCO.php`). El `iframe` se crea **sin atributo `sandbox`** (`player.php`) y el contenido se sirve del mismo origen (`pluginfile.php`). SCORM, por diseño, asume mismo origen y acceso al padre; aislarlo lo haría incompatible. Su defensa es **server-side**: `confirm_sesskey()` + capacidad `mod/scorm:savetrack` antes de guardar *tracking*. Riesgo: un SCORM malicioso ejecuta JS con el origen de Moodle (lee DOM y cabalga la sesión); la validación limita *qué acciones del servidor* puede forzar, no la lectura del contexto. Es **el menos aislado en el cliente** de los analizados. Limitación adicional del estándar: como el *tracking* ocurre en el cliente, el alumnado puede falsear `completion/score` con `LMSSetValue`, documentado desde hace años [16]; la integridad de la evaluación digital es un campo propio [17].

4.4 H5P (`mod_h5pactivity` / `core_h5p`)

H5P inyecta el contenido en un `iframe about:blank` mediante `contentDocument.write()`; ese documento **hereda el origen del padre** (no es opaco, **sin atributo `sandbox`**: `h5piframe.mustache:32-34`), así que su aislamiento **no** proviene del origen. Lo que controla es **qué** ejecuta, y aquí hay que distinguir dos planos.

Plano de los parámetros (control negativo). El texto de autoría de `content.json` pasa por `H5PContentValidator::validateText` aplica `filter_xss()` con una **lista cerrada de etiquetas** que **no incluye `<script>`** y `_filter_xss_attributes` descarta los atributos `on*` (`h5p.classes.php:4303-4384, :5033-5054`); sin tags, el campo se escapa con `htmlspecialchars`. Lo verificamos: `evil.h5p` inyecta `<script></img onerror>` en un campo de texto y H5P los descarta. Un `.h5p` que confíe en los **parámetros** para ejecutar JS es, por tanto, un **control negativo**. Aun así, el saneamiento por listas es **históricamente frágil** —las mutaciones de `innerHTML` (mXSS) y el XSS de cliente evaden filtros que no parsean el DOM [18], [19]—, de modo que la robustez no debe descansar solo en el filtro.

Plano de las librerías (la protección es la capacidad, no el saneamiento). Pero las librerías H5P son **código de confianza por diseño**: su `preloadedJs` se carga como `<script src=pluginfile.php/.../core_h5p/...>` y se ejecuta **same-origin** y **sin `sandbox`** en la página de Moodle (`player.php:484-500, h5p.js:391-437`). La documentación de H5P lo dice sin ambigüedad —“*JavaScript files ... are by default and necessity allowed for H5P libraries but not for H5P content*”— y recomienda que “*only trusted users should be given permission to update h5p libraries*” [20]. La única barrera para que el contenido de autor introduzca su **propia** librería es una **capacidad**, no un filtro: instalar una librería nueva desde un `.h5p` subido exige `moodle/h5p:updatelibraries` (arquetipo **manager**, `RISK_XSS`), evaluada **contra quien sube el fichero** (`api.php:403-405, helper.php:210-224, h5p.classes.php:1577-1579`). El profesorado solo tiene `moodle/h5p:deploy` (puede usar librerías ya instaladas, no instalar nuevas); un paquete que requiera una librería ausente se **rechaza en validación**. Construimos `evil-h5p-library.h5p` (la librería H5P.ExePocAlert, cuyo `preloadedJs` muestra un aviso y booleanos de capacidad de solo lectura). Que su `preloadedJs` corre **same-origin** y **sin `sandbox`** está **verificado sobre el código fuente** (rutas `archivo:línea` citadas) y el paquete está **validado estructuralmente**; la ejecución en vivo *end-to-end* se documenta como **procedimiento manual reproducible** (subida con rol de gestión → el aviso aparece al ver el contenido), ya que la **automatización headless** del *file picker* de Moodle 5 no resultó fiable y queda como trabajo pendiente de automatizar. Es exactamente el patrón de `mod_page`: la defensa real es la

capacidad/rol, no el saneamiento ni un sandbox —y el riesgo es de **confianza en la administración / cadena de suministro**. Este vector está acreditado en el mundo real: en Chamilo, importar un `.h5p` malicioso llegó a **RCE autenticado** (CVE-2026-30875) [21].

H5P tampoco es inmune por el lado del contenido: historial de XSS evadible —MDL-67110 (ejecución de JS) [22], CVE-2024-43439 (XSS reflejado vía mensaje de error de H5P, que esquivo el filtro de contenido) [23], CVE-2024-3111 (stored XSS vía subida de SVG hasta *backdoor* en el plugin H5P de WordPress) [24]—; y su `postMessage` valida `event.source` y `context==='h5p'` pero **no** `event.origin` y envía con `'*'` —el tipo de comprobación que [25] halló explotable en 84 sitios populares—.

Veredicto matizado: H5P no ejecuta el HTML/JS de los *parámetros* (los filtra: control negativo), pero las librerías son código de confianza que corre *same-origin* sin sandbox; lo que separa al contenido de autor de ejecutar JavaScript es la capacidad `moodle/h5p:updatelibraries` (gestión/administración), no el saneamiento. Mismo patrón que `mod_page`. Evidencia: `evidencias/resultados-h5p-library.json`; PoC: `poc/evil-h5p-library.h5p`.

4.5 eXeLearning en Moodle (`mod_exelearning`, `mod_exeweb`, `mod_exescorm`)

En las versiones estables, `.elpx` se extrae y se sirve por `pluginfile.php` y se muestra en un `iframe` con `sandbox="allow-scripts allow-same-origin allow-popups allow-forms allow-popups-to-escape-sandbox"`. Es el **mejor aislado de las tres integraciones eXe en Moodle** (bloquea `allow-top-navigation` y `allow-modals`), pero mantiene `allow-same-origin` por tres dependencias del puente SCORM síncrono (el padre lee `iframe.contentDocument` para mapear `objectid`; *pipwerks* recorre `window.parent`; el *teacher-mode* *hider* inyecta CSS en el `contentDocument`). Con ambos flags sobre contenido del propio origen, el **sandbox no aísla de verdad**. `mod_exeweb` muestra el contenido **sin sandbox** y `mod_exescorm` tampoco `sandboxea`: ambos son **más débiles**. Verificamos en vivo (modo *legacy*) que el contenido del `iframe` lee `parent.M.cfg.sesskey` y forja `core_user_update_users` (renombró una cuenta de laboratorio, revertida).

4.6 eXeLearning en WordPress y Omeka S

En las versiones estables, `wp-exelearning` embebe el paquete con `sandbox="allow-scripts allow-same-origin allow-popups"` → **mismo origen**; la sonda obtuvo `canAccessParent: true`, `canReadParentDocument: true` y localizó enlaces a `/wp-admin/`. Además, su proxy REST `/content/<hash>` tiene `permission_callback => '__return_true'` (lectura no autenticada del contenido por hash). En `omeka-s-exelearning`, la vista pública del ítem usó `sandbox="allow-same-origin allow-scripts allow-popups allow-popups-to-escape-sandbox"`; la sonda midió `canAccessParent: true`, `canReadParentCookie: true`, `isOpaqueOrigin: false` → **mismo origen** (Omeka sí impone validación CSRF obligatoria). WordPress y Omeka no tienen `sesskey`; usan *nonces* y *tokens* CSRF, pero la lógica es la misma: el riesgo no es que el contenido “vea” el *token*, sino que el servidor acepte una acción por venir con un *token* válido que un script *same-origin* puede leer del DOM.

5. Discusión

Implicaciones para docentes. El JS pegado de IA o copiado se ejecuta, en la mayoría de integraciones estables, con el origen del LMS y en la sesión de **cada** visitante. La interactividad legítima no exige confiar el origen al contenido.

Implicaciones para la administración. La superficie crítica es *quién* puede publicar HTML/JS (`mod/page:addinstance`, `moodle/site:trustcontent`) y *con qué aislamiento*. Moodle no emite CSP global por defecto; conviene activar el modo seguro de las integraciones que lo ofrecen y tratar los paquetes externos como no confiables. Conviene recordar que la *ausencia de síntomas no prueba seguridad*: un script malicioso *same-origin* puede permanecer latente y activarse solo cuando lo abre una persona con rol de administración (ataque dirigido y paciente, sección 4.2); la prevalencia del XSS persistente de cliente está documentada empíricamente [26]. El XSS en Moodle es objeto de auditoría continua [27], [28].

Impacto de la IA generativa (RQ4) — motivación, no medición. No medimos empíricamente la IA generativa; la tratamos como un factor que cambia el *modelo de amenaza operativo*: no introduce una vulnerabilidad nueva, pero hace más frecuente el patrón peligroso —contenido HTML/JS plausible, copiado sin revisar, publicado por un rol de confianza—, convirtiendo un riesgo latente en cotidiano. Es una hipótesis de plausibilidad y frecuencia, no un resultado cuantitativo; medirla (p. ej. con un estudio de uso) queda como trabajo futuro. CEDEC/INTEF

lo formula para los REA: un recurso con código de IA no revisado puede ser legalmente abierto pero pedagógica y técnicamente cerrado e inseguro [6].

Compatibilidad frente a seguridad. El dilema central del modo seguro: el origen opaco aísla, pero **rompe los embeds de terceros** que necesitan su propio origen (YouTube/Vimeo), porque el **sandbox** se propaga al iframe anidado. Resolver esto sin renunciar al aislamiento es el objeto de la sección 6.2–sección 6.3.

6. Mitigaciones

Separamos lo externo analizado (6.1) de la mitigación de aportación propia (6.2 —dividida en *requisitos de diseño*, 6.2.1, e *implementación de referencia* en eXeLearning, 6.2.2—) y de las mitigaciones propuestas (6.3).

6.1 Estado externo: el modelo de confianza en el autor

Moodle, en su núcleo, **confía en el autor** para los embeds: el filtro de medios reconoce proveedores conocidos por *allowlist* de URL (clases `core_media_player_external` para YouTube/Vimeo) y embebe el iframe **directamente, sin sandbox**; H5P confía en sus librerías curadas; SCORM confía en el paquete subido. Es un modelo razonable cuando la autoría es de confianza (profesorado), pero no aísla contenido no confiable.

6.2 Mitigación: requisitos de diseño e implementación de referencia

6.2.1 Requisitos de diseño (independientes de la implementación) Para servir HTML/JS de autor sin exponer la sesión, una implementación debería cumplir, con independencia de la plataforma:

1. **Origen opaco por defecto.** `sandbox` con `allow-scripts` (y `allow-popups/allow-forms` según necesidad) **sin `allow-same-origin`**; el modo aislado como predeterminado y cualquier modo *same-origin* solo como respaldo, con normalización *fail-safe* al modo seguro.
2. **Una única fuente de verdad** para los *tokens* del `sandbox` (un *helper*), en vez de cadenas duplicadas por plantilla.
3. **Directiva `sandbox` en la CSP de la respuesta** del contenido (no solo en el atributo del iframe): el documento conserva el origen opaco aunque se abra fuera del iframe (pestaña nueva, popup, URL cruda).
4. **CSP restrictiva:** `connect-src 'self'` (corta la exfiltración), `frame-ancestors 'self' / X-Frame-Options: SAMEORIGIN` (anti-*clickjacking*), `object-src 'none'`, `base-uri 'none'`, `form-action` acotado y `Permissions-Policy` que desactive cámara/micrófono/geolocalización/pago.
5. **Neutralización de los tipos activos del paquete** (SVG/XML) con una CSP sin scripts.
6. **Sin acceso directo padre iframe.** Donde haya que cruzar (modo docente, calificación SCORM), resolver en servidor (estado por la `src`) o por `postMessage` validado —identidad de ventana + `nonce` + lista cerrada de acciones— con las credenciales (`sesskey/nonce`) solo en el padre y revalidación *server-side*.
7. **Servir por capacidad de solo lectura:** un *token* que solo lea ficheros (no el `sesskey`) o un *content proxy* con `Content-Type` explícito y protección de *path-traversal*.
8. **Tests de seguridad:** que el `sandbox` esperado esté presente por modo, que el *handler* de mensajes rechace orígenes/fuentes desconocidos y que el modo por defecto sea el aislado.

6.2.2 Implementación de referencia en las integraciones de eXeLearning Las tres integraciones mantenidas (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`) instancian R1–R8 como su modo seguro activado por defecto —aportación propia del autor; véase la tabla de procedencia en la sección 9—. Lo verificamos en navegador en las tres (origen opaco confirmado en Chromium y Firefox 146/Gecko; el contenido benigno sigue renderizando). Medición antes/después: en *legacy* el contenido lee `parent.M.cfg.sesskey` y forja una acción; en *secure* el mismo intento lanza `SecurityError` (origen opaco). En Moodle, R7 se cumple con `tokenpluginfile` (un *token* que solo lee ficheros) y el puente SCORM (R6) transmite solo el *score* por `postMessage` validado, con el `sesskey` exclusivamente en el padre.

Tabla de amenazas (activo → condición → impacto → mitigación):

Activo	Amenaza	Condición	Impacto	Mitigación
Sesión del LMS	JS <i>same-origin</i> aprovecha la sesión	el recurso ejecuta JS de autor en el origen del LMS	acciones autenticadas (según el rol de quien visualiza)	origen opaco (sin <code>allow-same-origin</code>)

Activo	Amenaza	Condición	Impacto	Mitigación
DOM autenticado	lectura del padre desde el iframe	<code>allow-same-origin</code> presente	exposición de datos/nonce	<code>sandbox</code> sin <code>same-origin</code> + directiva <code>sandbox</code> en la respuesta
Tracking SCORM	manipulación cliente del <code>score</code>	API JS accesible <code>same-origin</code>	notas falseadas	puente <code>postMessage</code> validado + revalidación <i>server-side</i>
Token de ficheros	exfiltración del <code>token</code> de solo-lectura	CSP permite <code>img/script-src https:</code>	acceso temporal a ficheros del paquete	perfil de CSP estricta (propuesto, sección 6.3) + TTL corto
Persona usuaria privilegiada	carga latente y dirigida	el JS espera —o induce con un mensaje— a que un perfil de gestión/administración abra el recurso	acotado por las capacidades de ese perfil (creación de cursos, matriculación, etc.)	el origen opaco elimina el punto de apoyo; revisión por rol
Otros visitantes	propagación por mensaje	<code>core_message_send_instant_messages</code> (rol <code>user</code>)	mensaje mediante de interacción	contenido confinado al origen; revisión por rol

Limitación asumida. El origen opaco es incompatible con embeds de terceros que necesitan su propio origen (YouTube/Vimeo): el `sandbox` se propaga al reproductor anidado. Para no degradar la experiencia, el modo seguro renderiza el vídeo mediante un *overlay* mediado por el padre (el contenido pide promover un iframe; el padre, fuera del *sandbox*, valida y superpone el reproductor real). Implementación actual: *allowlist* de proveedores con reconstrucción de URL canónica. La generalización a cualquier proveedor se trata en la sección 6.3.

6.3 Mitigaciones propuestas (trabajo futuro)

- **Vídeo de cualquier proveedor sin lista blanca (invariante estructural).** En lugar de mantener una *allowlist* de hosts (YouTube/Vimeo) con reconstrucción por proveedor, promover cualquier iframe cuyo `src` sea `https + cross-origin` al LMS (rechazando *same-origin*/subdominio/IP/loopback/userinfo). El argumento de seguridad: un iframe **cross-origin** no puede leer el LMS (la SOP protege al padre), exactamente el modelo de confianza que Moodle ya usa para embeber YouTube; el invariante “cross-origin” sustituye a la lista de hosts y admite YouTube, Vimeo, Dailymotion, Mediateca de Madrid y cualquier proveedor **sin enumerarlos** ni crear subdominios. Contrapartida a documentar: el autor podría embeber cualquier contenido cross-origin (riesgo de *phishing*/tracking, no de escape); para despliegues de alta seguridad, una opción de “modo estricto” puede reactivar una lista. Alternativa más conservadora: **oEmbed en servidor** (el LMS pide el HTML de embed al proveedor), más seguro pero limitado a proveedores con oEmbed y con coste de *fetch* en servidor [29].
- **Perfil de CSP estricta opcional.** Cerrar el residual detectado: un script en el iframe opaco aún puede exfiltrar el `token` de ficheros vía `` porque `img-src/script-src/media-src` admiten `https:`. Un perfil opcional (admin, por defecto desactivado para no romper imágenes externas/MathJax/CDN) que limite esas directivas a los assets del paquete cierra el canal.
- **Defensas de plataforma complementarias.** Mecanismos *secure-by-default* aplicados por el navegador, como **Trusted Types**, han eliminado el DOM-XSS a escala en grandes bases de código [30]; son complementarios al aislamiento por origen opaco —restringen la *capacidad* peligrosa en el límite de la plataforma, la misma lógica que aplicamos a `mod_page` y a las librerías H5P—.
- **Configurabilidad del *helper* de embeds por la administración** (paridad entre las tres integraciones).

7. Limitaciones

Entorno local y desechable (no medimos producción); versiones concretas (los resultados se atan a los *commits* de la sección 3.1); sin explotación destructiva (demostramos la cadena, no el abuso); verificado en **dos motores** (Chromium y Firefox 146/Gecko), con **Safari/WebKit no probado** (trabajo futuro); el vector de librería H5P está verificado en código y con PoC validada, con la ejecución *end-to-end* como procedimiento manual (la automatización *headless* del *file picker* de Moodle 5 no fue fiable); `mod_exeweb/mod_exescorm` se infieren del código, no de prueba viva; la IA generativa (RQ4) no se mide. El modelo de amenaza se centra en el aislamiento cliente, no en toda la superficie del LMS. La generalización a otras versiones o configuraciones exige re-verificar contra el código correspondiente.

8. Ética y divulgación responsable

Los comportamientos descritos son, en su mayoría, documentados y por diseño: `mod_page` con `noclean=true`, el *same-origin* de SCORM y el modelo de confianza del filtro de medios y de las librerías H5P no son *0-day* de terceros, sino decisiones de diseño conocidas y *gated* por capacidad. El modo seguro de la sección 6.2 es una mitigación ya integrada en software del propio autor (aportación propia).

Divulgación responsable. No procedía una divulgación coordinada con terceros: (i) los comportamientos evaluados son documentados/por diseño, no defectos reportables, por lo que no se notificaron como vulnerabilidades; (ii) el único vínculo con un fallo de terceros es la cita de **CVE-2026-30875** (Chamilo), que ya era público y estaba parcheado *upstream* antes de este trabajo; (iii) no se halló ningún *0-day* de terceros sin parchear. Si en el futuro surgiera tal hallazgo, se coordinaría con los mantenedores antes de difundirlo. Publicamos PoC *redacted* (solo booleanos + errores), sin *payloads* reutilizables ni pasos de abuso, y los cambios reversibles de laboratorio se revirtieron.

9. Declaración de conflicto de interés

El autor es colaborador del proyecto eXeLearning y autor/mantenedor de varias piezas del ecosistema analizado. Este doble rol —analista y desarrollador de parte del objeto de estudio— se declara por transparencia; no invalida los resultados (verificables sobre el código y los artefactos), pero el lector debe conocerlo. Para separarlo sin ambigüedad, la siguiente **tabla de procedencia** distingue qué se evalúa, qué vínculo tiene el autor y con qué evidencia:

Componente	Vínculo del autor	Rol en el estudio	Evidencia
Moodle núcleo (<code>mod_page</code> , <code>mod_scorm</code>)	Ninguno (tercero)	Objeto evaluado	Vivo + código
H5P (<code>core_h5p</code>)	Ninguno (tercero)	Objeto evaluado	Parámetros: vivo · librería: código + PoC validada + manual
SCORM (estándar)	Ninguno (tercero)	Objeto evaluado	Vivo + código
<code>mod_exeweb</code> , <code>mod_exescorm</code>	Ecosistema eXeLearning (sin vínculo declarado)	Objeto evaluado	Solo código (inferencia)
<code>mod_exelearning</code> , <code>wp-exelearning</code> , <code>omeka-s-exelearning</code>	Autor/mantenedor	Objeto evaluado (estable) y mitigación propia (modo seguro, 6.2.2)	legacy : vivo · secure : vivo (Chromium + Firefox 146)
Safari / WebKit	—	Cobertura de navegador	Pendiente (trabajo futuro)

Las afirmaciones sobre software propio se sostienen sobre la misma evidencia citable (**archivo:línea**, JSON de evidencia, PoC) que las de terceros; ninguna mitigación propia se evalúa solo por inspección del autor.

10. Disponibilidad de artefactos

Se publica un paquete de artefactos reproducible en <https://github.com/ersec0/lms-untrusted-content-security-paper> (texto bajo **CC-BY-4.0**; código y PoC bajo **MIT**): la sonda `probe.js` (15 comprobaciones, *redacted*), los paquetes PoC y su `build.sh` —incluida la librería H5P `evil-h5p-library.h5p` que respalda (permite reproducir) la ejecución de `preloadedJs`—, la **matriz** completa por **archivo:línea** (**matriz-seguridad.md**), los **anexos** por plataforma/navegador (**anexos-tecnicos.md**), las evidencias en JSON (**evidencias/**, incluida la verificación cross-browser en Firefox de las tres integraciones y sus scripts de Playwright, y **resultados-h5p-library.json**) y el *script* de generación del documento, junto con una guía de reproducibilidad (**REPRODUCIBILITY.md**), un `Makefile` con los objetivos de construcción y las sumas de verificación de los PDF publicados (**pdf/SHA256SUMS**). Las versiones analizadas se identifican por los *commits* de la sección 3.1. Las PoC no contienen *payloads* reutilizables; los PDF de las fuentes con derechos de autor no se redistribuyen (se enlazan por DOI/URL).

11. Conclusiones

JavaScript no es el enemigo: el contenido educativo interactivo a menudo lo necesita. El riesgo nace de ejecutar JavaScript no confiable con el mismo origen que el LMS (RQ1–RQ2): de lo analizado, H5P es el más controlado por diseño (no ejecuta HTML de autor), `mod_page` está protegido por capacidad/rol (no por saneamiento), y SCORM/eXeLearning estable corren *same-origin* por compatibilidad. La respuesta a RQ3 es un patrón concreto

y verificado —origen opaco + `sandbox` estricto + CSP (incl. directiva en la respuesta) + puente `postMessage` validado— que mantiene interactividad y *tracking* sin exponer el DOM autenticado; ese patrón es ya el modo por defecto de las integraciones mantenidas de eXeLearning. Sobre RQ4, la IA generativa no crea el fallo y no la medimos: la planteamos como factor de frecuencia que vuelve cotidiano el patrón. Mantenemos separados el estado externo evaluado, la mitigación de aportación propia (secciones 6.2 y 9) y el trabajo futuro (sección 6.3), para que el lector distinga la evaluación del parche propio. La regla que lo resume: **aislar, validar y no confiar en el JavaScript del recurso como si fuera parte de la plataforma.**

Anexos técnicos (matriz completa por `archivo:línea`, PoC redacted, resultados por plataforma/navegador, limitaciones metodológicas): ver `anexos-tecnicos.md` y `matriz-seguridad.md`.

12. Declaración sobre el uso de IA generativa

En la preparación de este trabajo se utilizaron herramientas de IA generativa (asistentes de redacción y de codificación basados en modelos de lenguaje) como apoyo en la redacción y reestructuración del texto, en la generación y refactorización de las pruebas de concepto y los *scripts* de evidencia, y en la elaboración de tablas. La IA **no figura como autora**. El autor diseñó la investigación, definió la metodología, ejecutó y verificó todas las pruebas en el entorno de laboratorio, comprobó manualmente cada afirmación técnica, el código y las evidencias citadas, y asume la **responsabilidad plena** del contenido.

13. Referencias y estándares

- [1] N. Nikiforakis *et al.*, “You are what you include: Large-scale evaluation of remote JavaScript inclusions,” in *Proc. 2012 ACM conference on computer and communications security (CCS ’12)*, 2012, pp. 736–747. doi: [10.1145/2382196.2382274](https://doi.org/10.1145/2382196.2382274).
- [2] T. Lauinger, A. Chaabane, S. Arshad, W. Robertson, C. Wilson, and E. Kirda, “Thou shalt not depend on me: Analysing the use of outdated JavaScript libraries on the web,” in *Proc. 2017 network and distributed system security symposium (NDSS ’17)*, 2017. doi: [10.14722/ndss.2017.23414](https://doi.org/10.14722/ndss.2017.23414).
- [3] A. Khamparia and B. Pandey, “Threat driven modeling framework using petri nets for e-learning system,” *SpringerPlus*, vol. 5, no. 1, p. 446, 2016, doi: [10.1186/s40064-016-2101-0](https://doi.org/10.1186/s40064-016-2101-0).
- [4] A. J. J. Joseph and M. Mariappan, “Approaches to overcome security risks and threats in online learning applications,” in *Secure data management for online learning applications*, CRC Press, 2023. doi: [10.1201/9781003264538-3](https://doi.org/10.1201/9781003264538-3).
- [5] S. A.-L. Akacha and A. I. Awad, “Enhancing security and sustainability of e-learning software systems: A comprehensive vulnerability analysis and recommendations for stakeholders,” *Sustainability*, vol. 15, no. 19, p. 14132, 2023, doi: [10.3390/su151914132](https://doi.org/10.3390/su151914132).
- [6] CEDEC (Centro Nacional de Desarrollo Curricular en Sistemas no Propietarios), “Mantener la «a» de abierto en los REA en tiempos de IA.” Accessed: June 14, 2026. [Online]. Available: <https://cedec.intef.es/mantener-la-a-de-abierto-en-los-rea-en-tiempos-de-ia/>
- [7] MDN Web Docs, “The inline frame element: The sandbox attribute.” Accessed: June 13, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe#sandbox>
- [8] WHATWG, “HTML living standard — sandboxing.” Accessed: June 13, 2026. [Online]. Available: <https://html.spec.whatwg.org/multipage/browsers.html#sandboxing>
- [9] S. Stamm, B. Sterne, and G. Markham, “Reining in the web with content security policy,” in *Proceedings of the 19th international conference on world wide web (WWW)*, 2010, pp. 921–930. doi: [10.1145/1772690.1772784](https://doi.org/10.1145/1772690.1772784).
- [10] L. Weichselbaum, M. Spagnuolo, S. Lekies, and A. Janc, “CSP is dead, long live CSP! On the insecurity of whitelists and the future of content security policy,” in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security (CCS)*, 2016, pp. 1376–1387. doi: [10.1145/2976749.2978363](https://doi.org/10.1145/2976749.2978363).
- [11] S. Roth, T. Barron, S. Calzavara, N. Nikiforakis, and B. Stock, “Complex security policy? A longitudinal analysis of deployed content security policies,” in *Proc. 2020 network and distributed system security symposium (NDSS ’20)*, 2020. doi: [10.14722/ndss.2020.23046](https://doi.org/10.14722/ndss.2020.23046).
- [12] Advanced Distributed Learning (ADL), “SCORM 1.2 run-time environment,” Advanced Distributed Learning Initiative, 2001. Accessed: June 13, 2026. [Online]. Available: <https://adlnet.gov/projects/scorm/>

- [13] Advanced Distributed Learning (ADL), “SCORM 2004 4th edition — run-time environment (API API_1484_11),” Advanced Distributed Learning Initiative, 2009. Accessed: June 13, 2026. [Online]. Available: <https://adlnet.gov/projects/scorm/>
- [14] P. Hutchison, “SCORM API wrapper for JavaScript (pipwerks).” Accessed: June 13, 2026. [Online]. Available: <https://github.com/pipwerks/scorm-api-wrapper>
- [15] H5P Group, “H5P documentation — content types and libraries.” Accessed: June 13, 2026. [Online]. Available: <https://h5p.org/documentation>
- [16] P. Hutchison, “Cheating in SCORM.” Accessed: June 13, 2026. [Online]. Available: <https://pipwerks.com/2009/03/22/cheating-in-scorm/>
- [17] P. Dawson, *Defending assessment security in a digital world: Preventing e-cheating and supporting academic integrity in higher education*. Routledge, 2020. doi: 10.4324/9780429324178.
- [18] M. Heiderich, J. Schwenk, T. Frosch, J. Magazinius, and E. Z. Yang, “mXSS attacks: Attacking well-secured web-applications by using innerHTML mutations,” in *Proc. 2013 ACM SIGSAC conference on computer and communications security (CCS '13)*, 2013, pp. 777–788. doi: 10.1145/2508859.2516723.
- [19] B. Stock, S. Lekies, T. Mueller, P. Spiegel, and M. Johns, “Precise client-side protection against DOM-based cross-site scripting,” in *Proc. 23rd USENIX security symposium (USENIX security '14)*, 2014, pp. 655–670. Available: <https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-stock.pdf>
- [20] H5P, “Security (H5P documentation): Libraries are trusted code.” Accessed: June 14, 2026. [Online]. Available: <https://h5p.org/documentation/installation/security>
- [21] MITRE CVE, “CVE-2026-30875: Authenticated remote code execution in chamilo LMS via H5P package import.” Accessed: June 14, 2026. [Online]. Available: <https://github.com/chamilo/chamilo-lms/security/advisories/GHSA-mj4f-8fw2-hrfm>
- [22] Moodle Tracker, “MDL-67110: XSS in malicious seemingly H5P content.” Accessed: June 13, 2026. [Online]. Available: <https://tracker.moodle.org/browse/MDL-67110>
- [23] Deutsche Telekom Security and Moodle, “CVE-2024-43439: Reflected XSS in Moodle via H5P error message.” Accessed: June 13, 2026. [Online]. Available: <https://github.security.telekom.com/2024/08/reflected-xss-moodle.html>
- [24] CleanTalk PSC, “CVE-2024-3111: Interactive content – H5P (WordPress) stored XSS to backdoor creation.” Accessed: June 13, 2026. [Online]. Available: <https://research.cleantalk.org/cve-2024-3111/>
- [25] S. Son and V. Shmatikov, “The postman always rings twice: Attacking and defending postMessage in HTML5 websites,” in *Proceedings of the network and distributed system security symposium (NDSS)*, 2013. Available: <https://www.ndss-symposium.org/ndss2013/ndss-2013-programme/postman-always-rings-twice-attacking-and-defending-postmessage-html5-websites/>
- [26] M. Steffens, C. Rossow, M. Johns, and B. Stock, “Don’t trust the locals: Investigating the prevalence of persistent client-side cross-site scripting in the wild,” in *Proc. 2019 network and distributed system security symposium (NDSS '19)*, 2019. doi: 10.14722/ndss.2019.23009.
- [27] R. R. Al-azaiza and T. S. M. Barhoom, “Enhance MOODLE security against XSS vulnerabilities,” *International Journal of Computing and Digital Systems*, vol. 5, no. 5, pp. 421–430, 2016, doi: 10.12785/ijcds/050507.
- [28] Sonar, “Playing dominos with Moodle’s security.” Accessed: June 13, 2026. [Online]. Available: <https://www.sonarsource.com/blog/playing-dominos-with-moodles-security-2/>
- [29] oEmbed, “oEmbed — an open format for embedding content via a provider API.” <https://oembed.com/>.
- [30] P. Wang, B. Á. Gumundsson, and K. Kotowicz, “Adopting trusted types in production web frameworks to prevent DOM-based cross-site scripting: A case study,” in *2021 IEEE european symposium on security and privacy workshops (EuroS&PW)*, 2021, pp. 60–73. doi: 10.1109/EuroSPW54576.2021.00013.

Anexo A — Matriz de seguridad

Evidencia verificada contra el HEAD de cada repositorio (junio 2026). Cada celda “Cita” remite a `archivo:línea`. Las afirmaciones de comportamiento del navegador están marcadas [hecho] (verificado en código o prueba) o [inferencia].

SHAs de referencia: `mod_exelearning 2c5473d · mod_exeweb 60d24fb · mod_exescorm e985f4d · moodle 2104c372962 (5.x, code en public/) · exelearning 8101f54e · wp-exelearning 9eb07ff · omeka-s-exelearning 33faf89`.

1. Tabla resumida (para el cuerpo del artículo)

Plataforma / recurso	¿Ejecuta JS del autor?	¿Mismo origen que el LMS?	sandbox del iframe	Aislamiento real
mod_page (Página)	Sí (verificado en vivo: ejecuta <code><script></code>)	Sí	— (no es iframe)	Ninguno server-side (<code>noclean</code>); gated por capacidad (<code>RISK_XSS</code>)
mod_scorm (core)	Sí	Sí	ninguno	Ninguno (confía en el SCO)
mod_h5pactivity / core_h5p	Parámetros No / librerías Sí (<code>preloadedJs</code>)	Sí (<code>about:blank</code> hereda origen)	— (<code>contentDocument.write</code> , sin <code>sandbox</code>)	Parámetros filtrados por semántica; el JS de librería corre <i>same-origin</i> , gate <code>h5p:updatelibraries</code>
mod_exelearning	Sí	Sí	<code>allow-scripts</code> <code>allow-same-origin</code> <code>allow-popups allow-forms</code> <code>allow-popups-to-escape-sandbox</code>	Parcial (bloquea top-nav y modals)
mod_exeweb	Sí	Sí	ninguno	Ninguno
mod_exescorm	Sí	Sí	ninguno	Ninguno
wp-exelearning	Sí	Configurable: secure opaco (def.) / legacy same-origin	<code>secure: allow-scripts</code> <code>allow-popups · legacy: +</code> <code>allow-same-origin</code>	Fuerte en secure (origen opaco); parcial en legacy
omeka-s-exelearning	Sí	Configurable: secure opaco (def.) / legacy same-origin	<code>secure: allow-scripts</code> <code>allow-popups</code> <code>allow-popups-to-escape-sandbox</code> <code>· legacy: +</code> <code>allow-same-origin</code>	Fuerte en secure (origen opaco); ahora también h5p vistas públicas

Lectura rápida: *ejecutar JavaScript del autor no es el problema; el problema es ejecutarlo **con el mismo origen** que el LMS y **sin** una frontera explícita*. Las dos columnas que de verdad importan son “mismo origen” y “aislamiento real”.

1.1 Matriz de riesgo formal

Criterio de clasificación explícito (referenciado desde la sección 3.4 del artículo): siete dimensiones observables más la separación entre **impacto máximo demostrado** (verificado en laboratorio) e **impacto inferido** (deducido del modelo del navegador). “SS” = *server-side*.

Mecanismo	JS autor	Mismo origen	Rol p/ publicar	Rol visitante	Token en DOM	CSP restr.	Mutación SS alcanzable (rol)	Impacto máx. demostrado	Impacto inferido
mod_page	Sí	Sí (top)	profesor/gestor (addinstance)	cualquiera	Sí (<code>sesskey</code>)	No	Sí (servicios AJAX)	autoedición del propio perfil (vivo)	acciones acotadas por el rol del visitante
mod_scorm	Sí	Sí	profesor	cualquiera	Sí	No	Sí (gated <code>savetrack</code>)	lectura de DOM/ <code>sesskey</code> (vivo)	forja acotada por validación SS
H5P · parámetros	No (filtrado)	Sí	—	cualquiera	n/a	No	n/a	control negativo (vivo)	—
H5P · librería	Sí (<code>preloadedJs</code>)	Sí	manager (<code>updatelibraries</code>)	cualquiera	Sí	No	Sí	ruta en código + PoC validada (manual)	JS arbitrario <i>same-origin</i>
mod_exelearning (legacy)	Sí	Sí	profesor	cualquiera	Sí	No	Sí	lee <code>sesskey</code> y forja (vivo)	escalado por rol
mod_exelearning (secure)	Sí	No (opaco)	profesor	cualquiera	No (token solo-lectura)	Sí	solo vía puente validado	SecurityError (vivo Chromium+FF146)	aislado
mod_exeweb / mod_exescorm	Sí	Sí	profesor	cualquiera	Sí	No	Sí	— (solo código)	acceso total <i>same-origin</i>

Mecanismo	JS autor	Mismo origen	Rol p/publicar	Rol visitante	Token en DOM	CSP restr.	Mutación SS alcanzable (rol)	Impacto máx. demostrado	Impacto inferido
wp-exelearning / omeka-s-exelearning (secure)	Sí	No (opaco)	autor/editor	cualquiera	No	Sí	—	opaco (vivo, Chromium+FF146)	aislado

2. Matriz técnica completa (anexo)

2.1 mod_exelearning (2c5473d)

Atributo	Valor	Cita
Cómo se publica	El profesor sube un .elpx; se extrae y se sirve por pluginfile.php	lib.php:513-568
HTML/JS arbitrario	Sí (HTML/JS del autor del paquete)	view.php:446-447
JS se ejecuta	Sí	iframe allow-scripts
Mismo origen	Sí (\$CFG->wwwroot único)	view.php:446-447
Usa iframe	Sí, #exelearningobject	view.php:446-447
sandbox	allow-scripts allow-same-origin allow-popups allow-forms allow-popups-to-escape-sandbox	view.php:446-447
allow-top-navigation	No (omitido a propósito)	view.php:446-447
allow-popups-to-escape-sandbox	Sí (residuo a quitar)	view.php:446-447
CSP	No emitida	lib.php:513-568
Permissions-Policy	No emitida (pendiente)	lib.php:513-568
X-Frame-Options	No (en pluginfile); core pone sameorigin en páginas	weblib.php:1604-1605
postMessage	Sí, solo en el editor (no en el visor)	amd/src/editor_modal.js:441-449
Valida event.origin	Sí (editor)	editor_modal.js:441-449
Valida event.source	Sí (=== iframe.contentWindow)	editor_modal.js:441-449
Token/contexto	sesskey validado server-side en track.php	track.php:40, view.php:387-390
Acceso a parent	Sí (mismo origen)	bridge SCORM
Acceso a cookies no-HttpOnly	Sí (mismo origen)	mismo origen
Acceso al sesskey	Sí (va en la URL de track.php, legible desde el DOM/JS)	view.php:387-390
Localiza formularios edición	Sí, si el DOM padre los tiene	mismo origen
Cambios reales	No demostrado (server valida sesskey + capability)	track.php:40
Mitigaciones	Sandbox parcial; require_capability en pluginfile; require_sesskey server-side	lib.php:513-568, track.php:40
Riesgos que quedan	XSS cross-component same-origin (riesgo residual)	—
Impacto de IA sin revisar	Alto: JS generado se ejecuta con el origen de Moodle	—

Dependencias **duras** de same-origin (impiden quitar allow-same-origin sin reescribir el bridge): 1. El padre lee `iframe.contentDocument` para mapear `objectid` → `js/scorm_tracker.js:75-86` y `:151-154`. 2. El hijo (pipwerks) recorre `window.parent` buscando `window.API` → `assets/scorm/SCORM_API_wrapper.js:62-118`. 3. El `teacher-mode hider` inyecta CSS en `contentDocument` → `classes/local/ui/teacher_mode_hider.php:44-61`.

2.2 mod_exweb (60d24fb) y mod_exescorm (e985f4d)

Atributo	mod_exweb	mod_exescorm
iframe	embed_general.mustache:32-34	player.php:283-285 (noscript)
sandbox	ninguno	ninguno
HTML/JS arbitrario	Sí	Sí
Mismo origen	Sí	Sí
SCORM API walk	No (sin tracking)	Sí (SCORM_API_wrapper.js:71-78, 140-142)
sesskey	—	URL param (datamodels/scorm_12.js:19-24)
CSP en pluginfile	No (lib.php:448-512)	No (lib.php:1064-1142)
teacher-mode hider	Sí (locallib.php:90-107)	—

2.3 Moodle core: mod_scorm (2104c372962)

Atributo	Valor	Cita
iframe del SCO	sin sandbox	public/mod/scorm/player.php:279-285
API discovery	myFindAPI recorre win.parent y window.opener	loadSCO.php:113-122, :128-129
sesskey	confirm_sesskey() antes de guardar	datamodel.php:53
Capability	mod/scorm:savetrack	datamodel.php:56
Mismo origen	Si (wwwroot/pluginfile.php)	locallib.php:2323, 2327

Veredicto: mod_scorm corre HTML/JS no confiable **sin sandbox** — más débil que mod_exelearning. Su defensa es server-side (sesskey + capability), no aislamiento del cliente.

2.4 Moodle core: core_h5p / mod_h5pactivity (2104c372962)

Atributo	Valor	Cita
iframe	src="about:blank"	h5piframe.mustache:33
Construcción	contentDocument.write(...)	h5p.js:390-394
Origen del contenido	hereda el del padre (same-origin) [hecho]	h5p.js:391-394 (nota verificada)
HTML/JS arbitrario (parámetros)	No: content.json validado contra la semántica; filter_xss sin <script> ni on*	h5p.classes.php:2260-2282 (filterParameters), :4303-4384, :5033-5054
HTML/JS de librería (preloadedJs)	Sí: código de confianza, <script src=pluginfile.php/.../core_h5p> same-origin y sin sandbox	player.php:484-500, h5p.js:391-437
Gate de instalación de librería	moodle/h5p:updatelibraries (arquetipo manager , RISK_XSS), evaluada contra quien sube; profesorado solo h5p:deploy	lib/db/access.php, helper.php:210-224, api.php:403-405, h5p.classes.php:1577-1579
Whitelist	Extensiones curadas (content + librería)	framework.php:698-700
postMessage handler	Valida event.source===window.parent y event.data.context==='h5p'; no valida event.origin	embed.js:38-46, h5p.js:457-465
postMessage envió	targetOrigin = '*' (origen-agnóstico)	embed.js:64-73, h5p.js:484-493
\$CFG->h5pcrossorigin	CORS para media (img/audio/video), no para el iframe	helper.php:383, config-dist.php:778-786

Veredicto (matizado): hay que distinguir dos planos. En los **parámetros** de contenido, H5P **no ejecuta HTML/JS de autor:** filtra el texto con filter_xss contra una lista cerrada de etiquetas sin <script> y descarta on* (h5p.classes.php:4303-4384, :5033-5054) — control negativo, más controlado por diseño que un .elpx. En el plano de las **librerías**, en cambio, **sí:** el preloadedJs de una librería es **código de confianza** que corre *same-origin* y sin sandbox (player.php:484-500, h5p.js:391-437); la barrera para que el autor introduzca su propia librería es la **capacidad** moodle/h5p:updatelibraries (gestión/administración, RISK_XSS), no el saneamiento — **mismo patrón que mod_page**. PoC: poc/evil-h5p-library.h5p; evidencia: evidencias/resultados-h5p-library.json. Además **no es inmune** por contenido: XSS documentados (MDL-67110, CVE-2024-43439 —XSS reflejado vía mensaje de error—, CVE-2024-3111 —stored XSS vía SVG—) y precedente de RCE por import de .h5p (CVE-2026-30875, Chamilo). Su postMessage no valida event.origin (mitiga con event.source + context), patrón a no copiar a ciegas.

Acción privilegiada (Moodle, verificado en código): el contenido same-origin (Página, .elpx, SCO) lee window.M.cfg.sesskey y puede invocar servicios web **AJAX** como core_user_update_users ('ajax' => true, cap moodle/user:update, public/lib/db/services.php:1982-1989) → si lo abre un usuario privilegiado, puede forjar la edición de una cuenta. La defensa es server-side (capacidad + validación), no ocultar el token.

2.5 Moodle core: mod_page + filtrado global (2104c372962)

Atributo	Valor	Cita
Render	format_text(..., noclean=true)	public/mod/page/view.php:90-93
trusttext por defecto	0 (desactivado)	locallib.php:53
¿trusttext activo?	!empty(\$CFG->enabletrusttext)	weblib.php:958-961
¿texto confiable?	activo y capability	weblib.php:949-950
Capability	moodle/site:trustcontent (RISK_XSS, solo profesor/manager)	db/access.php:452-454
noclean en salida	\$formatoptions->noclean = true	public/mod/page/lib.php:352
purify_html (otros contextos)	HTMLPurifier quita <script> en texto no noclean	weblib.php:1105-1107
enabletrusttext por defecto	0 (desactivado)	public/admin/settings/security.php:68
X-Frame-Options	sameorigin salvo \$CFG->allowframing	weblib.php:1604-1605
CSP global	ninguna por defecto	weblib.php (NOT FOUND)

Veredicto (verificado en vivo): en `mod_page`, `<script>` y `<img onerror>` se ejecutan — `noclean=true` hace que `format_text` traduzca a `clean=false` y **no** pase por `HTMLPurifier`. El filtrado `purify_html` aplica a otros campos de texto no confiable (sin `noclean`), **no** a la salida de `mod_page`. La protección de `mod_page` es la **capacidad** de crear/editar la Página (`mod/page:addinstance`; HTML confiable ligado a `moodle/site:trustcontent`, `RISK_XSS`), no el saneado. `enabletrusttext=0` por defecto **no** impide la ejecución porque `mod_page` usa `noclean`.

2.6 eXeLearning core / contenido exportado (8101f54e)

Atributo	Valor	Cita
SCORM API walk	<code>window.parent (500) + win.top.opener</code> , sin validación de origen	<code>SCORM_API_wrapper.js:71-77, :136-141</code>
localStorage	clave <code>exeTeacherMode</code> (revela contenido de profesor)	<code>exe_export.js:100-130</code>
postMessage listener <code>eval()</code>	valida type, no <code>event.origin</code> sí (callbacks de carga de script, tooltips, onclick)	<code>asset_url_resolver.js:905-906</code> <code>common.js:805,810,757;</code> <code>common_edition.js:39</code>
MathJax	local empaquetado (no CDN)	<code>common.js:26-115</code>
iframes exportados	sin sandbox	<code>asset_url_resolver.js:933-936</code>
Validación server-side	ninguna (HTML exportado es estático/self-contained)	—

2.7 wp-exelearning (9eb07ff)

Atributo	Valor	Cita
Modo iframe	ajuste <code>exelearning_iframe_sandbox_mode: secure (def.) / legacy</code> ; helper único, <i>fail-safe</i> a <code>secure</code>	<code>includes/class-iframe-sandbox.php</code>
iframe (secure)	<code>sandbox="allow-scripts allow-popups" (sin same-origin → opaco) + referrerpolicy="no-referrer"</code>	<code>public/class-shortcodes.php (sandbox_tokens())</code>
iframe (legacy)	<code>sandbox="allow-scripts allow-same-origin allow-popups" (same-origin)</code>	<code>class-iframe-sandbox.php (TOKENS_LEGACY)</code>
Teacher mode (secure)	server-side por la <code>src</code> (<code>exe-teacher/exe-teacher-toggler</code>), aplicado por el content proxy	<code>includes/class-content-proxy.php</code>
Content proxy	<code>permission_callback => '_return_true'</code> (lectura no autenticada por hash SHA1)	<code>includes/class-exelearning-rest-api.php:44-50</code>
Nonce en guardado	usa <code>permission_callback</code> (capability), no <code>wp_verify_nonce</code>	<code>rest-api.php:223</code>
Nonce en editor	<code>wp_verify_nonce</code> al cargar página	<code>class-exelearning-editor.php:111</code>

2.8 omeka-s-exelearning (33faf89)

Atributo	Valor	Cita
Modo iframe	ajuste <code>exelearning_iframe_mode: secure (def.) / legacy</code> ; helper único, <i>fail-safe</i> a <code>secure</code>	<code>src/Service/IframeSandbox.php</code>
iframe (secure)	<code>sandbox="allow-scripts allow-popups allow-popups-to-escape-sandbox" (sin same-origin → opaco); también en las dos vistas públicas</code> (antes fijadas a <code>allow-same-origin</code>)	<code>ExeLearningRenderer.php, view/.../public/*-show.phtml</code>
iframe (legacy) <code>src</code>	añade <code>allow-same-origin</code> fijado por JS desde <code>window.location</code> (prefijo de base)	<code>IframeSandbox::tokens()</code> <code>ExeLearningRenderer.php</code>
CSRF <code>postMessage</code>	token obligatorio (rechaza vacío/nulo) mixto: <code>editor.js:83</code> usa <code>'*'</code> ; <code>omeka-exe-download.js:122-125</code> y <code>omeka-exe-bridge.js:46</code> usan origen específico	<code>src/Controller/CsrfValidationTrait.php:24-38</code> citas

3. Mitigaciones emparejadas (riesgo → mitigación → limitación)

Riesgo	Mitigación	Qué no resuelve
Lectura de cookies de sesión	<code>HttpOnly</code> + <code>SameSite=Strict</code>	No protege frente a XSS en el mismo origen ni a tokens expuestos en el DOM

Riesgo	Mitigación	Qué no resuelve
Acceso al parent / mismo origen	iframe de origen opaco sin allow-same-origin (o srcdoc , o subdominio)	Rompe el bridge SCORM directo (window.API); exige bridge postMessage
Localización/envío de formularios con sesskey /nonce postMessage no validado	Validación server-side del token por acción + capacidades mínimas Allowlist cerrada + validación de event.origin y event.source	Si el server confía en el cliente, ocultar el token en el DOM no sirve Una allowlist mal mantenida reabre la superficie
Ejecución de JS arbitrario	CSP restrictiva + sandbox sin allow-same-origin	El contenido legítimo que necesita JS (SCORM/H5P) exige arquitectura de bridge explícita
Navegación superior / popups	Omitir allow-top-navigation y allow-popups-to-escape-sandbox	Puede degradar UX de contenido legítimo que abre recursos externos

Principio rector: la mitigación efectiva vive en el **servidor** y en las **cabeceras**, no en confiar en que el cliente “no encuentre” el token.

4. Tabla de concienciación (perfil no técnico)

Riesgo detectado	Por qué importa	Mitigación	Limitación de la mitigación
El recurso puede leer la cookie de sesión	Permitiría suplantar al usuario autenticado	HttpOnly + SameSite=Strict	No cubre XSS en el mismo origen
El recurso accede al marco padre	Acceso al DOM autenticado del LMS	Origen opaco sin allow-same-origin	Rompe SCORM directo; exige bridge
El recurso localiza el sesskey /nonce	Primer paso para forzar acciones	Validación server-side por acción	Inútil si el server confía en el cliente
El recurso localiza formularios de edición	Permitiría crear/modificar contenido	Capacidades mínimas + validación server-side	Depende de la configuración de roles
postMessage sin validar origen/source	Canal de control no autenticado	Allowlist + validación origin/source	Allowlist mal mantenida reabre el hueco

Anexo B — Anexos técnicos

Material de soporte del artículo. Para la matriz completa por **archivo:línea**, ver **matriz-seguridad.md**. Aquí: metodología, PoC redacted, resultados por plataforma/navegador, comportamiento del navegador, limitaciones y trabajo futuro.

A. Metodología

1. **Verificación de código (estática).** Barrido read-only de los 10 repositorios contra su HEAD actual (work-flow paralelo de 10 agentes). Cada afirmación se ancla a **archivo:línea** + **sha**. Las versiones y SHAs analizados se listan en la sección 3.1 del artículo (Metodología).
2. **Prueba viva (dinámica).** Entornos Docker locales y desechables. Sonda inyectada en el iframe del contenido y lectura de booleanos. Navegador: **dos motores** —Chromium y **Firefox 146 (Gecko)**— vía Playwright (**evidencias/resultados-firefox*.json**).
3. **Separación hechos / inferencias.** [**hecho**] = verificado en código o prueba; [**inferencia**] = deducción del comportamiento estándar del navegador.

B. La sonda (probe.js) — redacted

15 comprobaciones, salida solo booleana + nombre de error. Reglas duras: sin red, sin POST, sin lectura de valores reales, sin mutadores SCORM, popup abierto y cerrado al instante.

Fragmentos representativos (el código completo está en **poc/probe.js**):

```
// Solo se registra el NOMBRE del error, nunca el mensaje (podría llevar valores).
function errName(e){ return e && (e.name || 'Error'); }
```

```
// Cookie del padre: solo se comprueba SI es legible; el valor nunca se guarda ni se imprime.
try {
  var c = window.parent.document.cookie;      // lanza SecurityError si cross-origin/opaco
  R.canReadParentCookie = (typeof c === 'string');
```

```

} catch (e) { R.errors.cookie = errName(e); }
R.parentCookieValue = 'REDACTED';

// SCORM: se DETECTA el objeto API recorriendo parents; NUNCA se invoca un método.
while (win && tries < 20) {
  if (win.API_1484_11) { api = win.API_1484_11; break; }
  if (win.API) { api = win.API; break; }
  win = win.parent; tries++;
}
R.canCallScormApi = !!api; // reachable; deliberadamente NO se llama

```

Las cuatro PoC se generan de forma reproducible con poc/build.sh (ver poc/README.md).

C. Resultados por plataforma

Plataforma	Modo	iframe sandbox (runtime/código)	same-origin	Resultado
mod_exelearning	vivo	allow-scripts allow-same-origin allow-popups allow-forms allow-popups-to-escape-sandbox	sí	acceso al padre/cookie/sesskey/forms true; canCallScormApi:true (1.2)
mod_scorm (core)	vivo	sin sandbox (scorm_object)	sí	acceso total same-origin; canReachScormApi:true (1.2)
mod_h5pactivity	vivo	iframe externo sin sandbox; interno about:blank (hereda origen)	sí	frame interno same-origin (canReadTopDocument:true); parámetros de contenido no ejecutan (filtrados), pero el preloadedJs de una librería sí corre same-origin (gate h5p:updatelibraries) <script> y EJECUTADOS; sin saneo server-side (noclean); gated por capacidad
mod_page	vivo	n/a (no iframe)	sí	canAccessParent/Document/Cookie:true; canCallScormApi:false; eval no bloqueado canAccessParent/Document:true; localiza /wp-admin/; eval no bloqueado esperado: acceso total same-origin
Omeka S (vista pública)	vivo	allow-same-origin allow-scripts allow-popups allow-popups-to-escape-sandbox	sí	
wp-exelearning	vivo	allow-scripts allow-same-origin allow-popups	sí	
mod_exeweb / mod_exescorm	código	sin sandbox	sí	

“vivo” = ejecutado en el navegador en esta sesión (Moodle 5.0.7 local). “código” = verificado en fuente; el resultado de la sonda se deduce del sandbox/origen.

Resultado vivo Omeka (crudo)

```

{"platform":"omeka-s public item view",
  "iframeSandbox":"allow-same-origin allow-scripts allow-popups allow-popups-to-escape-sandbox",
  "injectedProbe":{"isOpaqueOrigin":false,"canAccessParent":true,"canReadParentDocument":true,
    "canReadParentCookie":true,"canFindCsrf":false,"canFindEditForms":false,
    "canUseLocalStorage":true,"canUsePostMessage":true,"canCallScormApi":false}}

```

(También en evidencias/resultados-vivos.json; captura en evidencias/01-omeka-item-view.png.)

D. Resultados por navegador

- **Chrome/Chromium** (probado): el iframe con allow-scripts + allow-same-origin sobre contenido del propio origen permite el acceso al padre y muestra el aviso conocido de que puede “escapar” del sandbox. srcdoc + sandbox sin allow-same-origin → origen opaco, acceso al padre bloqueado (SecurityError).

- **Firefox 146 (Gecko)** (probado) — *[hecho]*: replicamos la comprobación con Playwright (evidencias/firefox-isolation/firefox-moodle-test.cjs). El resultado es **idéntico al de Chromium**: el iframe con `allow-same-origin` lee el padre; sin él, el documento es **opaco** y el acceso lanza `SecurityError`. Los embeds reales en modo seguro de `mod_exelearning` (servido por `tokenpluginfile`), `wp-exelearning` y `omeka-s-exelearning` resultan **opacos** también en Firefox (`contentWindow` lanza `SecurityError` en las tres) — UA `...rv:146.0 Gecko/20100101 Firefox/146.0`; datos en `resultados-firefox.json` y `resultados-firefox-moodle.json`.
- **Safari / WebKit** (no probado) — *[inferencia]*: el modelo de orígenes y el atributo `sandbox` están estandarizados (HTML Living Standard), por lo que se espera un comportamiento de aislamiento equivalente; diferencias menores posibles en `SameSite` por defecto y en políticas de cookies de terceros. Marcado como trabajo futuro.

E. Comportamiento del navegador (referencia)

- **Cookies HttpOnly** — no aparecen en `document.cookie`; un script (aunque sea same-origin) no las lee. Reducen el impacto de `canReadParentCookie: true`, pero no protegen tokens que estén en el DOM.
- **SameSite** — `Strict/Lax` limitan el envío de cookies en peticiones cross-site; `None` (o ausencia, según navegador) las envía. No sustituyen al aislamiento por origen.
- **sesskey / nonce / CSRF token** — su seguridad depende de la **validación server-side por acción**, no de ocultarlos. Un script same-origin que los lee del DOM no logra nada si el servidor revalida capacidad + token.
- **sandbox sin allow-same-origin** → **origen opaco**: `document.cookie` vacío/inaccesible útil, `localStorage` lanza o queda aislado por origen opaco, sin acceso a `parent.document`.
- **sandbox con allow-same-origin + allow-scripts** → el aislamiento es nominal: el contenido del propio origen puede manipular su relación con el padre.
- **postMessage mal validado** — aceptar mensajes sin comprobar `event.origin` y `event.source` convierte el puente en un canal de control para cualquier ventana.
- **localStorage en origen opaco** — particionado/!inaccesible; no se comparte con el origen real de la plataforma.

F. Compatibilidad

- **SCORM** — asume mismo origen y descubrimiento de `window.API` por recorrido de `parent`. Aislar por origen exige un puente `postMessage` que emule el contrato **síncrono** de pipwerks (`cmi{}` local en el hijo, `flush` en `beforeunload/LMSFinish`).
- **H5P** — ya usa `postMessage` (aunque sin validar `event.origin`); su seguridad no depende del origen sino del contenido curado + `filterParameters`.

G. Mitigaciones emparejadas (riesgo → mitigación → limitación)

Riesgo	Mitigación	No resuelve
Lectura de cookies de sesión Acceso al <code>parent</code> / mismo origen	<code>HttpOnly</code> + <code>SameSite=Strict</code> Origen opaco / <code>srcdoc</code> / subdominio	XSS same-origin; tokens en el DOM Rompe SCORM directo; exige puente <code>postMessage</code>
Formularios con <code>sesskey/nonce</code>	Validación server-side por acción + capacidades	Nada si el servidor confía en el cliente
<code>postMessage</code> no validado JS arbitrario	Allowlist + <code>event.origin</code> y <code>event.source</code> CSP + <code>sandbox</code> sin <code>allow-same-origin</code>	Allowlist mal mantenida SCORM/H5P legítimos exigen arquitectura de puente
Navegación superior / popups	Omitir <code>allow-top-navigation</code> / <code>allow-popups-to-escape-sandbox</code>	Puede degradar UX de contenido legítimo

Principio rector: la mitigación efectiva vive en el **servidor** y en las **cabeceras**.

H. Limitaciones metodológicas

- Prueba viva ejecutada en `mod_exelearning`, `mod_scorm`, `mod_h5pactivity`, `mod_page` (Moodle 5.0.7 local), `wp-exelearning` (WordPress wp-env) y Omeka S. Solo `mod_exeweb/mod_exescorm` quedan verificados en código; su resultado de sonda es equivalente a casos ya ejecutados (same-origin, sin `sandbox`).

- En **wp-exelearning** el **evil.elpx** se publicó vía **wp media import** + el comando del plugin **wp exelearning reprocess** (extracción) y un bloque **exelearning/elp-upload server-rendered**; el fichero temporal se eliminó tras la prueba.
- Las pruebas Moodle se hicieron como **administrador**; **canFindSesskey/canFindCourseEditForms** reflejan ese rol. Con rol estudiante, **canFindCourseEditForms** sería **false** (el resto del acceso same-origin se mantiene).
- **Gotcha de entorno observado**: el *version sentinel* 999999999 del plugin hace que Moodle no detecte cambio de versión y **no actualice el esquema** de **mdl_exelearning**; con volúmenes sembrados por una versión previa, una consulta a la columna nueva (**completionstatusrequired**) lanza **dml_read_exception** al reconstruir la caché de un curso con actividades eXeLearning. Las pruebas de **mod_page** se hicieron por ello en un curso limpio sin actividades eXeLearning.
- Dos motores probados (Chromium y **Firefox 146/Gecko**, vía Playwright; **evidencias/resultados-firefox*.json**). Safari/WebKit: inferencia por estándar (trabajo futuro).
- Versiones concretas (ver SHAs). El comportamiento puede cambiar entre versiones; toda cita es reproducible contra el **sha** indicado.
- La PoC mide **capacidad**, no **explotación**: detecta que el acceso es posible; no demuestra un exploit funcional (por diseño ético).

I. Trabajo futuro

- Completar la prueba viva de **mod_exelearning** + SCORM/H5P/Page nativos y de **wp-exelearning**.
- Repetir en **Safari/WebKit** (Firefox 146/Gecko ya verificado).
- **Automatizar la verificación end-to-end del vector de librería H5P**: el *file picker* de Moodle 5 no se automatiza de forma fiable en *headless*, por lo que la ejecución de **preloadedJs** se confirma hoy con un procedimiento manual reproducible (**evidencias/resultados-h5p-library.json**).
- Evaluar el modo seguro **iframe mode** (origen opaco + puente **postMessage**) end-to-end con la suite de tracking.
- Medir el impacto del *toggle* CSP estricto-con-excepción para contenido externo (MathJax, YouTube).

J. Notas de seguridad de esta investigación (checklist cumplido)

- ☒ Sin cookies/tokens/**sesskey** reales en logs ni en el artículo.
- ☒ Sin endpoints externos ni código de exfiltración.
- ☒ **La sonda distribuida (probe.js) no hace POST ni red** (solo detecta capacidades). Las confirmaciones de impacto (anexo siguiente) usaron POST reales **autorizados y reversibles** sobre cuentas propias/de laboratorio, nunca destructivos ni en producción.
- ☒ Entornos locales y desechables; nada de producción.
- ☒ PoC didácticas e inocuas (solo booleanos + error redacted).
- ☒ Repos de plugin no modificados ni commiteados.

Anexo — Confirmación en vivo en una instalación de Moodle en línea (2026-06-13)

Prueba **autorizada por la persona propietaria** de la cuenta sobre su **propio perfil** en una instalación de Moodle en línea de pruebas (HTTPS; host y cuenta anonimizados). El recurso no confiable se empaquetó como **SCORM** y se abrió con una **cuenta de prueba con rol de profesorado** en el curso. No se atacó a terceros ni se escaló privilegio; el único cambio fue la foto de la propia cuenta (reversible).

Aislamiento observado

El SCORM corre **same-origin** (origen no opaco): el contenido leyó y modificó **window.parent.document** (intercambio de avatar en el DOM) y localizó el **sesskey** del padre. Confirma en un servidor real la brecha descrita para SCORM sin sandbox.

Frontera de capacidades (el modelo de seguridad aguanta para terceros)

- **core_user_update_users** (vía **/lib/ajax/service.php, ajax=true**) → **rechazado**: un profesor no tiene **moodle/user:update**.

- `/user/editadvanced.php?id=5` (edición de **admin**) → **sin formulario**: un profesor no puede abrir la edición avanzada.

Es decir: un usuario no privilegiado **no puede mutar a otros usuarios**.

Lo que sí funciona: autoedición del propio perfil (nombre + foto)

Cualquier usuario autenticado puede cambiar **su propio** nombre y foto. Se confirmó la **autoedición persistente del propio perfil** mediante el **reenvío del formulario legítimo de edición de usuario** de Moodle: un script *same-origin* obtiene el **sesskey** del DOM, sube una imagen al área *draft* del propio perfil y reenvía el **FormData** del formulario real —que ya incluye el **sesskey** y los campos ocultos—, sobrescribiendo nombre y foto. (*Evidencia conceptual: no se listan aquí los **endpoints** ni los nombres de campo concretos; la técnica es un reenvío del propio formulario, sin escalada de privilegio.*) La operación se realizó sobre una **cuenta propia de laboratorio** y se **revirtió**.

Conclusión para el artículo

El riesgo **escala con el privilegio de quien abre el recurso**: - **Alumno/profesor**: no puede tocar a terceros, pero **sí** puede ser inducido a cambiar su **propio** nombre y foto de forma **silenciosa** (auto-suplantación / desfiguración del propio perfil), sin interacción. - **Administrador**: la misma técnica, vía `core_user_update_users` o `editadvanced.php`, permitiría **mutar a otros usuarios**.

Mitigación verificada: el **modo iframe seguro** (origen opaco) sirve el recurso con **origen opaco** → leer `window.parent/M.cfg.sesskey` lanza `SecurityError`, lo que **corta toda la cadena** (el contenido ya no comparte origen ni sesión con el LMS).

Evidencia estructurada: `evidencias/resultados-moodle-online.json`. Flujo capturado con las Dev-Tools del navegador (network): subida al *draft* (200) → reenvío del formulario de usuario (200) → redirección de éxito al perfil.